Les bases de JavaScript

SOMMAIRE

Préliminaires

Partie 1 : L'algorithmie avec JavaScript

I) Les variables

1) Les variables de type « simple »
2) Utiliser des opérateurs
3) Les variables de type Object
4) Regrouper des données grâce aux tableaux
5) Distinguer les copies par valeur et par référence

II) La logique de programmation

1) Contrôler du code grâce aux conditions
2) Répéter du code grâce aux boucles

III) Organiser le code

1) Utiliser les fonctions
2) Coder proprement
3) Maîtriser la portée des variables
4) Les modules JavaScript : import / export

Partie 2 : Faire interagir JavaScript avec une page HTML

I) Manipuler les éléments d'une page web

1) La structure d'une page web : le DOM
2) Récupérer un élément du DOM

II) Modifier un élément d'une page web

1) Modifier un élément du DOM
2) Utiliser l'inspecteur pour vérifier les modifications

III) Créer un nouvel élément dans une page web

1) Créer un élément avec createElement
2) Créer plusieurs éléments avec innerHTML (interpolation)

IV) Interagir avec une page web grâce aux événements

1) La programmation événementielle
2) Écouter des événements avec addEventListener
3) Récupérer les informations sur un événement avec la variable event

Partie 3 : Aller plus loin avec JavaScript

I) La POO en JavaScript

1) Définir une classe
2) Hériter d'une classe

II) Communiquer avec un serveur : fetch et AJAX

1) Récupérer des données avec une requête AJAX en GET
2) Envoyer des données avec une requête AJAX en POST
3) Le format JSON

Préliminaires

Pour exécuter du code JavaScript, il faut :

- créer un fichier `.js` (par exemple `script.js`)
- ajouter dans le `<head>` du fichier HTML une balise `<script>` indiquant le chemin du fichier JS :

```
<script src="script.js"></script>
```

Il est aussi possible de mettre du code JS directement dans le fichier HTML dans une balise `<script></script>`, mais cela n'est pas une bonne pratique pour la lisibilité et la maintenance.

I) LES VARIABLES

1) Les variables de type « simple »

Déclarer une variable en JavaScript

```
let monAge = 42
```

On peut faire évoluer cette valeur en écrivant : `monAge = 43`

On peut aussi déclarer des constantes qui ne peuvent plus être changées par la suite :

```
const monPrenom = "David"
```

> ***Remarque :*** *mettre « ; » pour indiquer la fin d'une ligne de code n'est pas obligatoire en JS. « var » peut être utilisée pour déclarer une variable, mais elle est aujourd'hui obsolète.*

Déboguer son code avec console.log()

Pour vérifier le contenu d'une variable, il est possible d'utiliser l'instruction `console.log()` :

```
let monAge = 42
console.log(monAge)
```

Le résultat de `console.log()` se lit dans le navigateur :

- Inspecter (ou F12)
- onglet « console »

Les types de données basiques

- **String** : correspond à une chaîne de caractères, c'est-à-dire un texte
- **number** : correspond à un nombre
- **boolean** : correspond à un booléen (il n'accepte que deux valeurs : `true` ou `false`)

2) Utiliser des opérateurs

Manipuler des chiffres

```
const entreesPremiereSemaine = 1000
const entreesDeuxiemeSemaine = 2000
const totalEntrees = entreesPremiereSemaine + entreesDeuxiemeSemaine

let placesDejaVendues = 150
placesDejaVendues = placesDejaVendues + 10
// ou
placesDejaVendues += 10
```

On peut utiliser les opérateurs `+`, `-`, `/`, `*` ou les formes abrégées `+=`, `-=`, `*=`, `/=`

Manipuler des chaînes de caractères avec la concaténation

```
let premierePartie = "Mon nom est Bond"
let deuxiemePartie = ", James Bond."
let punchline = premierePartie + deuxiemePartie
// punchline vaut "Mon nom est Bond, James Bond."

// ou
let punchline = "Mon nom est Bond"
punchline += ", James Bond."
```

Les caractères `//` permettent d'insérer des commentaires dans le code.

Attention à bien différencier les types de données :

```
let placesDejaVendues = "150"
placesDejaVendues += "10"
console.log(placesDejaVendues)
// affichera "15010" et pas "160"
```

On résout le problème avec la fonction `Number()` :

```
let placesDejaVendues = Number("150")
placesDejaVendues += Number("10")
console.log(placesDejaVendues)
```

Manipuler des chaînes de caractères avec l'interpolation

Grâce aux backticks ` (AltGr + 7), on peut **interpoler** des variables avec `${ }` directement dans une chaîne de caractère :

```
let nom = "Bond"
let prenom = "James"
let punchline = `Mon nom est ${nom}, ${prenom} ${nom}.`
// punchline vaut "Mon nom est Bond, James Bond."
```

3) Les variables de type Object

Un **Object** JavaScript est un **conteneur** composé de **propriétés** qui ont chacune une valeur. On peut ainsi stocker plusieurs valeurs en même temps.

Déclarer un objet en JavaScript

Pour déclarer un objet en JavaScript, on utilise les accolades `{ }` :

```
let personnage = {
  nom: "Wayne",
  prenom: "Bruce",
  age: 35,
}
```

L'objet `personnage` à 3 propriétés (nom, prenom et age) qui possèdent chacune une valeur.

Manipuler les objets

Ajouter (ou remplacer) une propriété

```
personnage.couleurPrefere = "rouge"
```

Accéder à une propriété

```
console.log(personnage.nom)
// affichera "Wayne"
```

4) Regrouper des données grâce aux tableaux

Un **tableau** en JavaScript est un **objet** qui permet de lister plusieurs variables ou valeurs, et de les regrouper.

Déclarer un tableau en JavaScript

```
const maCollectionDeFilms = ["Titanic", "Jurassic Park"]

// On peut aussi stocker des variables :
const monFilmPrefere = "Titanic"
const monPremierFilm = "Jurassic Park"
const maCollectionDeFilms = [monFilmPrefere, monPremierFilm]
```

Accéder à un élément du tableau

```
let premierFilmDuTableau = maCollectionDeFilms[0]
// 0 = premier élément, 1 = second, etc.
```

Compter le nombre d'éléments

```
const nombreDeFilms = maCollectionDeFilms.length
console.log(nombreDeFilms)
// nombreDeFilms vaut 2
```

Modifier un tableau

push – Ajouter des données :

```
let mesFilms = ["Titanic", "Jurassic Park"]
mesFilms.push("Retour vers le futur")
// mesFilms vaut ["Titanic", "Jurassic Park", "Retour vers le futur"]
```

pop – Supprimer le dernier élément :

```
let mesFilms = ["Titanic", "Jurassic Park", "Retour vers le futur"]
mesFilms.pop()
// mesFilms vaut ["Titanic", "Jurassic Park"]
```

5) Distinguer les copies par valeur et par référence

La copie par valeur

```
let variableSimple1 = 25
let variableSimple2 = variableSimple1
```

Si on change la valeur de `variableSimple1`, alors `variableSimple1` et `variableSimple2` auront des valeurs différentes.

La copie par référence

```
let variableObjet1 = ['pomme', 'cerise']
let variableObjet2 = variableObjet1
```

Ici, si on change la valeur de `variableObjet1`, alors `variableObjet2` sera également modifiée.

Dans la copie par valeur, la mémoire est dupliquée. Dans la copie par référence, c'est une étiquette pointant vers le même emplacement mémoire qui est dupliquée.

> ***Remarque :*** *Les tableaux sont souvent déclarés avec const en JavaScript, même s'ils évoluent. Ce qui est constant, c'est l'endroit en mémoire où est stocké le tableau, pas son contenu.*

```
const fruits = ["pomme", "banane"]
fruits.push("cerise") // ok
fruits = ["kiwi"]     // ❌ erreur
```

II) LA LOGIQUE DE PROGRAMMATION

Un **algorithme** est une façon de concevoir les étapes d'un code dans le but de **résoudre un problème donné**.

Les **blocs de code** sont des regroupements de lignes de code délimités par des accolades `{ }`.

1) Contrôler du code grâce aux conditions

La condition if / else

```
if (condition) {
  // Code exécuté si la condition est vraie
} else {
  // Code exécuté si la condition est fausse
}
```

La condition peut être un booléen ou une comparaison :

```
let estMajeur = true
if (estMajeur) {
  console.log("Vous êtes majeur")
} else {
  console.log("Vous êtes mineur")
}
```

```
let age = 20
if (age >= 18) {
  console.log("Vous êtes majeur")
} else {
  console.log("Vous êtes mineur")
}
```

Les conditions if / else if / else et switch / case

Lorsque l'on doit gérer plusieurs réponses, on peut utiliser `if/else if/else` :

```
if (age >= 65) {
  console.log("Vous êtes sénior")
} else if (age >= 18) {
  console.log("Vous êtes majeur")
} else {
  console.log("Vous êtes mineur")
}
```

Structure conditionnelle `switch/case` (switch : valeur à tester, case : valeurs possibles) :

```
let note = "A"
switch (note) {
  case "A":
    console.log("Excellent travail !")
    break
  case "B":
    console.log("Bon travail !")
    break
  case "C":
    console.log("Travail correct !")
    break
  case "D":
    console.log("Insuffisant !")
    break
  default:
    console.log("Note invalide")
}
```

Remarque : *break sert à quitter le switch après être entré dans une condition. Il est impératif de ne pas l'oublier.*

2) Répéter du code grâce aux boucles

La boucle for

Utilisée lorsque l'on sait d'avance combien de fois il faudra répéter le code :

```
for (let compteur = 0; compteur < 3; compteur = compteur + 1) {
  console.log(compteur)
}
```

La boucle while

Utilisée pour répéter du code autant de fois qu'il le faut pour qu'une condition ne soit plus vraie :

```
let motDePasse = "Bon travail !"
let bonMotDePasse = "toto123"
while (motDePasse !== bonMotDePasse) {
  motDePasse = prompt("Entrez votre mot de passe.")
}
console.log("Mot de passe correct !")
```

La boucle forEach

Utilisée pour parcourir chaque élément d'un tableau :

```
let etudiants = ["Alice", "Karim", "Sofia", "Lucas"]
etudiants.forEach(function(etudiant) {
  console.log("Bonjour " + etudiant)
})
```

III) ORGANISER LE CODE

1) Utiliser les fonctions

Une **fonction** est un bloc de code que l'on peut appeler pour exécuter le code qu'elle contient. Elle peut effectuer une action, retourner quelque chose, et prendre en entrée 0, 1 ou plusieurs paramètres.

```
function saluer(nom) {
  console.log('Hello ' + nom)
}
saluer('Jean')
// Affichera : Hello Jean

function additionner(a, b) {
  return a + b
}
console.log(additionner(5, 3))
// Affichera : 8
```

Les fonctions fléchées

Les fonctions fléchées ont été introduites pour **simplifier la syntaxe** des fonctions :

```
const saluer = (nom) => {
  console.log('Hello ' + nom)
}

const additionner = (a, b) => {
  return a + b
}
```

Les deux notations sont équivalentes, c'est juste la syntaxe qui change.

> ***Remarque :*** *S'il n'y a qu'un seul paramètre, on peut se passer des parenthèses. S'il n'y a qu'une seule instruction, on peut se passer des accolades.*

```
const saluer = nom => console.log('Hello ' + nom)
const additionner = (a, b) => a + b  // pas de return dans la forme courte
```

Les fonctions anonymes

Les fonctions anonymes n'ont pas de nom. On en utilise par exemple dans forEach :

```
let etudiants = ["Alice", "Karim", "Sofia", "Lucas"]
etudiants.forEach(function(etudiant) {
  console.log("Bonjour " + etudiant)
})

// Elles peuvent aussi être fléchées :
etudiants.forEach((etudiant) => {
  console.log("Bonjour " + etudiant)
})

// ou en forme courte :
etudiants.forEach((etudiant) => console.log("Bonjour " + etudiant))
```

2) Coder proprement

Il est très important de respecter quelques bonnes pratiques :

- Définir une seule tâche spécifique pour chaque fonction (afficher, calculer, lancer une opération, etc.)
- Écrire des fonctions courtes (30 lignes max)
- Nommer clairement les fonctions et leurs paramètres (valable aussi pour les variables)

Découper le code en plusieurs fichiers pour une meilleure organisation du code :

```
<script src="config.js"></script>
<script src="script.js"></script>
```

3) Maîtriser la portée des variables

Les variables ont une **portée** définie par l'endroit où elles sont déclarées. On distingue :

- **Variables globales** : visibles et utilisables dans l'ensemble du code
- **Variables locales** : utilisables uniquement au sein du bloc de code `{ }` dans lequel elles sont définies

```
let monNombre = 1  // variable globale

function afficheUnNombre() {
  let monNombreLocal = 2  // variable locale
  console.log(monNombre)       // accessible
  console.log(monNombreLocal)  // accessible
}

console.log(monNombre)       // Affichera : 1
console.log(monNombreLocal)  // ReferenceError : non définie
afficheUnNombre()            // Affichera : 1 et 2
```

4) Les modules JavaScript : import / export

Les **modules** permettent de séparer le code en plusieurs fichiers et de réutiliser des fonctions, objets ou classes facilement.

On déclare les fonctions comme exportables avec `export` :

```
// Dans math.js
export const additionner = (a, b) => a + b;
export const multiplier = (a, b) => a * b;
```

Puis on les importe dans un autre fichier avec `import` :

```
// Dans app.js
import { additionner, multiplier } from './math.js';
console.log(additionner(5, 3));  // 8
console.log(multiplier(5, 3));   // 15
```

Partie 2 : Faire interagir JavaScript avec une page HTML

I) MANIPULER LES ÉLÉMENTS D'UNE PAGE WEB

JavaScript permet de **manipuler directement une page HTML** pour la rendre **interactive**.

1) La structure d'une page web : le DOM

Généralement, une page web est constituée de deux parties :

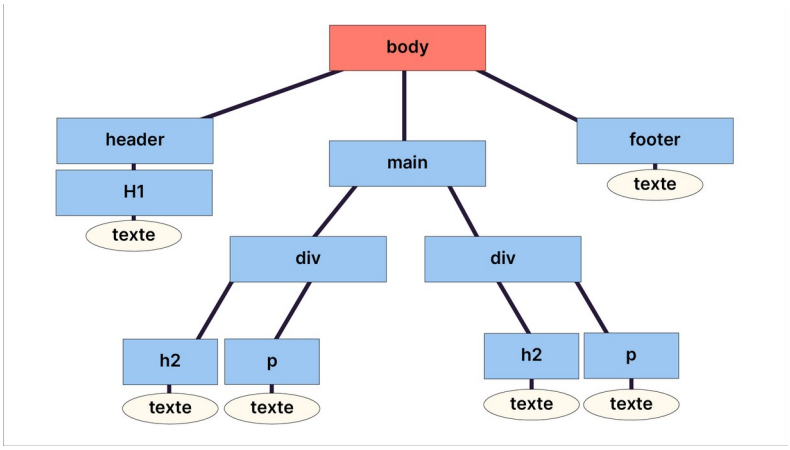
- **head** : informations générales sur la page (title, meta, link, style, script)
- **body** : contient ce qui est visible sur la page

Exemple de body :

```
<body>
  <header>
    <h1>AzerType</h1>
  </header>
  <main>
    <div>
      <h2>Le jeu</h2>
      <p>Un petit texte</p>
    </div>
  </main>
  <footer>@Copyright</footer>
</body>
```

JavaScript voit une page HTML comme une structure de nœuds appelée le **DOM** (Document Object Model) :

- le `<body>` est un nœud du DOM
- il mène à des nœuds enfants : `<header>`, `<main>`, `<footer>`
- chaque branche se termine généralement par un nœud texte



Pour JavaScript, chaque balise HTML est représentée par un objet **HTMLElement** qui possède des propriétés (id, class, contenu, etc.) et des méthodes.

2) Récupérer un élément du DOM

Utiliser defer pour différer l'exécution du script

Pour manipuler le DOM, JavaScript utilise une variable globale, `document`, qui représente la page HTML entière. Or, le script étant lancé dans la balise `head`, avant le `body`, la page HTML n'est pas encore chargée entièrement lorsque le script JS est appelé.

Pour que le script s'exécute après le chargement complet de la page, on ajoute `defer` :

```
<script src="script.js" defer></script>
```

Les méthodes de récupération

- **getElementById()** : récupère un élément via son id
- **querySelector()** : récupère le premier élément correspondant à un sélecteur CSS
- **querySelectorAll()** : récupère tous les éléments correspondants dans une NodeList
- **getElementsByClassName()** : récupère tous les éléments d'une classe dans une NodeList

getElementById

```
<div id="voiture">Peugeot</div>

let baliseVoiture = document.getElementById("voiture")
console.log(baliseVoiture)
// Affiche l'objet HTMLElement

// Accéder à une propriété du nœud :
console.log(baliseVoiture.clientHeight)
```

Le console.log montre qu'on a bien récupéré notre objet HTMLElement :

Si on clique sur le petit triangle à côté de cette div pour déployer le contenu, les détails de cet objet HTMLElement s'affichent :

```
▶ attributes: NamedNodeMap [ id="voiture" ]
  autocapitalize: ""
  autocorrect: true
  autofocus: false
  baseURI: "http://localhost:8080/"
  childElementCount: 0
▶ childNodes: NodeList [ #text ]
▶ children: HTMLCollection { length: 0 }
▶ classList: DOMTokenList []
  className: ""
  clientHeight: 10
  clientLeft: 0
  clientTop: 0
  clientWidth: 1904
  contentEditable: "inherit"
  currentCSSZoom: 1
▶ dataset: DOMStringMap(0)
  dir: ""
  draggable: false
  enterKeyHint: ""
▶ firstChild: #text "Peugeot"
```

querySelector

```
<div id="voiture"><span>Cachalot</span></div>

let baliseSpanVoiture = document.querySelector("#voiture span")
console.log(baliseSpanVoiture)
```

```
▼ <span>
    accessKey: ""
    accessKeyLabel: ""
```

querySelectorAll

```
<div class="choixCouleur">
  <input type="radio" name="couleur" id="rouge" value="1" checked>
  <input type="radio" name="couleur" id="bleu" value="2">
</div>
```

```
let listeInputRadio = document.querySelectorAll(".choixCouleur input")


// Parcourir la NodeList :
for (let i = 0; i <
listeInputRadio.length; i++) {
  console.log(listeInputRadio[i])
}
```

```
NodeList [ input#rouge , input#bleu ]
  0: <input id="rouge" type="radio" name="couleur" value="1" checked=""> 
  1: <input id="bleu" type="radio" name="couleur" value="2"> 
  length: 2
  <prototype>: NodeListPrototype { item: item(), keys: keys(), values: values(), … }
```

getElementsByClassName

```
let listeProduit = document.getElementsByClassName("produit")
console.log(listeProduit)
```

```
HTMLCollection { 0: div.produit , 1: div.produit , 2: div.produit , length: 3 }
```

II) MODIFIER UN ÉLÉMENT D'UNE PAGE WEB

1) Modifier un élément du DOM

JavaScript propose plusieurs méthodes pour modifier les éléments du DOM :

- **setAttribute()** : créer ou modifier un attribut
- **removeAttribute()** : supprimer un attribut
- **classList** : modifier une classe
- **innerHTML** : récupérer ou remplacer tout le contenu HTML
- **textContent** : récupérer ou modifier le texte d'un élément

setAttribute

```
<img id="premiereImage" src="image.jpg" alt="Ceci est une image de test">

let baliseImage = document.getElementById("premiereImage")
baliseImage.setAttribute("alt", "Ceci est une photo de profil")
// Le contenu de l'attribut alt de l'image a été modifié
```

```
// Pour les attributs HTML standards, on peut écrire directement :
baliseImage.alt = "Ceci est une photo de profil"

// Pour créer un attribut :
baliseImage.setAttribute("class", "photo")
```

removeAttribute

```
let baliseImage = document.getElementById("premiereImage")
baliseImage.removeAttribute("alt")
// L'attribut alt de l'image est supprimé et l'élément n'a plus de texte alternatif
```

classList

Plus pratique que `setAttribute` pour manipuler les classes :

```
<div class="listeMots centree actif photo"></div>

baliseImage.classList.add("nouvelleClasse")
baliseImage.classList.remove("photo")
```

innerHTML

```
<div id="monDiv">
 <p>Bonjour !</p>
</div>

let contenu = document.getElementById("monDiv").innerHTML
console.log(contenu)  // Affichera : <p>Bonjour !</p>

let contenu = document.getElementById("monDiv")
contenu.innerHTML = "<h1>Bonjour !</h1>"  // Affichera : <h1>Bonjour !</h1>
```

textContent

```
let texte = document.getElementById("monDiv").textContent
console.log(texte)  // Affichera : Bonjour !

let texte = document.getElementById("monDiv")
texte.textContent = "Hello !"  // Affichera : Hello !
```

2) Utiliser l'inspecteur pour vérifier les modifications

Pour vérifier les modifications sur un élément :

- Placer la souris sur l'élément dans le navigateur
- Faire un clic droit, puis « Inspecter »
- Vérifier dans la console que l'élément a bien été modifié

III) CRÉER UN NOUVEL ÉLÉMENT DANS UNE PAGE WEB

Pour créer des éléments, JavaScript propose deux approches principales :

- La méthode `createElement()`
- La propriété `innerHTML`

1) Créer un élément avec createElement

```
let nouvelElement = document.createElement("h1")
nouvelElement.textContent = "je suis un nouvel élément"

// Insérer nouvelElement dans le DOM en enfant de parentElement:
let parentElement = document.getElementById("conteneur")
parentElement.appendChild(nouvelElement)

// Résultat HTML :
<div id="conteneur"><h1>je suis un nouvel élément</h1></div>
```

Autres méthodes d'insertion :

- `prepend` : ajouter au début
- `before` : ajouter avant
- `insertAdjacentElement` : ajouter à un endroit précis

2) Créer plusieurs éléments avec innerHTML (interpolation)

La méthode `createElement` devient difficile pour de nombreux éléments imbriqués. On utilise alors l'**interpolation** qui tient compte des sauts de lignes et des tabulations en injectant les variables :

```
let contenuTitre = "Azertype"
let contenuParagraphe = "L'application pour apprendre à taper plus vite !"

let div = `
  <div>
    <h1>${contenuTitre}</h1>
    <p>${contenuParagraphe}</p>
  </div>
`


let body = document.querySelector("body")
body.innerHTML = div
```

IV) INTERAGIR AVEC UNE PAGE WEB GRÂCE AUX ÉVÉNEMENTS

1) La programmation événementielle

Jusqu'à présent, le code s'est toujours exécuté de manière **séquentielle** : les instructions s'exécutent les unes après les autres.

La **programmation événementielle** consiste à exécuter du code au moment où un événement donné se produit. Un **événement** correspond à une action spécifique : clic sur un bouton, frappe d'une touche du clavier, etc.

En pratique :

- Demander à JavaScript d'écouter l'événement avec un **eventListener**
- Puis lier l'événement à un bloc de code

2) Écouter des événements avec addEventListener

`addEventListener()` est une méthode qui prend deux paramètres :

- Le nom de l'événement (`click`, `keypress`, `submit`, `input`, etc.)
- Une fonction appelée lorsque l'événement se déclenche

```
// Dans le fichier HTML :
<button id="monBouton">Cliquez-moi !</button>

// Dans le fichier JavaScript :
let monBouton = document.getElementById("monBouton")
monBouton.addEventListener("click", function () {
  console.log("Vous avez cliqué sur le bouton")
})
```

```
// ou avec une fonction fléchée :
monBouton.addEventListener("click", () => {
  console.log("Vous avez cliqué sur le bouton")
})
```

3) Récupérer les informations sur un événement avec la variable event

On peut avoir besoin des informations sur l'événement, pour cela on utilise l'objet `event` :

```
document.addEventListener('keydown', (event) => {
  if (event.key === "a") {
    console.log("La touche A a été pressée")
  }
})
```

`event` est un objet JS contenant toutes les informations liées à l'événement. Propriétés et méthodes utiles :

- `event.target` : élément HTML qui a déclenché l'événement
- `event.type` : type d'événement (click, keydown, etc.)
- `event.key` : touche pressée pour les événements clavier
- `event.clientX` et `event.clientY` : coordonnées de la souris
- `preventDefault()` : empêche le comportement par défaut du navigateur
- `stopPropagation()` : empêche un événement de remonter aux éléments parents

Partie 3 : Aller plus loin avec JavaScript

I) LA POO EN JAVASCRIPT

Les **classes** sont une manière moderne et lisible de créer des objets et de gérer l'**héritage** en JavaScript.

1) Définir une classe

On crée un modèle d'objet avec le mot-clé `class` :

```
class Personne {
  #age  // propriété privée

  constructor(nom, age) {
    this.nom = nom;  // propriété publique
    this.#age = age;
  }

  sePresenter() {
    console.log(`Je suis ${this.nom} et j'ai ${this.#age} ans.`);
  }

  getAge() { return this.#age }
  setAge(nouvelAge) { this.#age = nouvelAge }
}

const jean = new Personne('Jean', 30);
jean.sePresenter();       // Je suis Jean et j'ai 30 ans.
console.log(jean.nom);    // Jean
console.log(jean.getAge()); // 30
// console.log(jean.#age); // ❌ Erreur : propriété privée
```

> ***Remarque :*** *JS ne supporte pas les attributs protégés nativement. Les méthodes peuvent être définies comme privées avec un # devant leur nom.*

2) Hériter d'une classe

`extends` permet à une classe d'hériter des propriétés et méthodes d'une autre classe. `super()` appelle le constructor de la classe parente :

```
class Employe extends Personne {
  constructor(nom, age, poste) {
    super(nom, age);  // appel du constructor de Personne
    this.poste = poste;
  }

  travailler() {
    console.log(`${this.nom} travaille en tant que ${this.poste}.`);
  }
}

const karim = new Employe('Karim', 28, 'développeur');
karim.sePresenter();  // hérité de Personne
karim.travailler();   // spécifique à Employe
```

II) COMMUNIQUER AVEC UN SERVEUR : FETCH ET AJAX

`fetch` est une fonction JavaScript qui permet d'envoyer une requête HTTP à un serveur et de récupérer sa réponse de manière **asynchrone** (sans recharger la page). On parle alors de requête **AJAX**.

Les requêtes AJAX sont très pratiques pour charger ou mettre à jour des données dynamiquement (liker un contenu, ajouter un produit dans un panier, etc.).

1) Récupérer des données avec une requête AJAX en GET

```
fetch('https://monsite/data')           // 1. Contacte le serveur
  .then(response => response.json())    // 2. Convertit la réponse en objet JS
  .then(data => console.log(data))      // 3. Utilise les données
  .catch(error => console.error(error));// 4. Gère les erreurs
```

2) Envoyer des données avec une requête AJAX en POST

```
fetch('https://api.example.com/utilisateurs', {
  method: 'POST',                                 // 1. Méthode POST
  headers: { 'Content-Type': 'application/json' },// 2. configuration du format JSON
  body: JSON.stringify({ nom: 'Alice' })          // 3. objet converti en texte json
})
  .then(response => response.json())              // 4. Réponse en objet JS
  .then(data => console.log(data))                // 5. Utilise la réponse
  .catch(error => console.error(error));          // 6. Gère les erreurs
```

3) Le format JSON

JSON (JavaScript Object Notation) est un format de texte permettant de transmettre des données entre un serveur et un navigateur. Il est simple, lisible et compris par presque tous les langages.

Il stocke des paires clé/valeur et peut contenir des objets `{ }`, des tableaux `[ ]`, des booléens et des dates :

```
{
  "utilisateur": {
    "nom": "Alice",
    "age": 25,
    "adresse": {
      "rue": "123 rue des Fleurs",
      "ville": "Paris",
      "codePostal": "75001"
    },
    "loisirs": ["lecture", "natation", "jeux vidéo"]
  },
  "actif": true,
  "derniereConnexion": "2026-03-13T10:30:00Z"
}
```